PDB Pathologies & Orthogonal Pattern Databases

Advanced Topics in Abstraction Heuristics

Lecture 5: Machine Intelligence (Part 2)

January 4, 2026

Contents

1 Pattern Database (PDB) Heuristics: The Intuitive Guide **2**
- 1.1 The Big Picture: What Are PDBs Really? . . . 2
- 1.2 How PDBs Work: The Three-Step Process . . . 3
- 1.3 Key Insight: Why Bigger Patterns = Better (But Slower) . . . 4
- 1.4 The Hash Function: How to Actually Look Things Up . . . 5
- 1.5 Real Example: Package Delivery . . . 6

2 Pathological Abstractions **9**
- 2.1 The Trade-off Challenge . . . 9
- 2.2 Pathological Case 1: One-State Abstraction . . . 10
- 2.3 Pathological Case 2: Identity Abstraction . . . 11
- 2.4 Pathological Case 3: Perfect Abstraction . . . 12
- 2.5 Summary of Pathological Cases . . . 13

3 Orthogonal Pattern Databases **14**
- 3.1 Motivation: Combining Multiple PDBs . . . 14
- 3.2 Maximizing vs. Summing Heuristics . . . 14
- 3.3 Actions Affecting Abstractions . . . 15
- 3.4 Orthogonal Abstractions: The Core Definition . . . 16
- 3.5 Orthogonal Patterns in FDR Tasks . . . 17
- 3.6 Understanding Orthogonality: Key Questions . . . 18
- 3.7 Examples of Orthogonal and Non-Orthogonal Patterns . . . 19
- 3.8 The Additivity Theorem . . . 21
- 3.9 Orthogonal Patterns are Additive . . . 22
- 3.10 Pattern Collections and The Cliques Heuristic . . . 23
- 3.11 Computing the Cliques Heuristic . . . 24
- 3.12 Worked Example: Simple Pattern Collection . . . 25
- 3.13 Example: Australia Driving Problem . . . 27
- 3.14 Limitations of the Cliques Heuristic . . . 28

4 Summary and Key Takeaways **30**
4.1 Main Concepts Summary 30
4.2 Quick Reference Guide 32
4.3 Practice Problems 33
4.4 Further Reading and Advanced Topics 34

1 Pattern Database (PDB) Heuristics: The Intuitive Guide

1.1 The Big Picture: What Are PDBs Really?

PDBs in Plain English

Core Idea: When a problem has too many variables to track, focus on a SUBSET of variables that matter most, precompute optimal solutions for just those variables, and use that as your heuristic.

Think of it like:

- **15-puzzle:** Instead of tracking all 15 tiles, track just tiles 1-4. Precompute how many moves it takes to get JUST those 4 tiles to their goal positions (ignoring the other 11).
- **Delivery problem:** Instead of tracking "where is the package + where are both trucks," just track "where is the package?" Simpler problem = easier to solve completely.

Why this works: The answer to the simpler problem is ALWAYS $\leq$ the answer to the full problem (admissible!). If getting just the package to the right place takes 3 moves, the full problem needs AT LEAST 3 moves.

1.2 How PDBs Work: The Three-Step Process

Step-by-Step PDB Construction

Step 1: Pick Your Pattern (Choose Variables to Care About)
You have a problem with variables {package, truck_A, truck_B}.
You decide: "I only care about the **package** location."
This is your **pattern** $P = \{\text{package}\}$.

Step 2: Create the Simplified Problem

- Throw away truck_A and truck_B from your state description
- Actions that ONLY moved trucks? They become self-loops (do nothing)
- Actions that affected the package? Keep their package effects
- **Result:** A tiny problem with 4 states: package at {L, R, A, B}

Step 3: Solve It Completely (Build the Database)

- Run backward search from all goal states
- For EVERY possible package location, compute: "What's the MINIMUM moves to get package to goal?"
- Store this in a lookup table (the "database")

Package at	L	A	B	R (goal)
Min moves needed	2	1	1	0

During Search:
You're at state (package=L, truck_A=R, truck_B=R).
To get heuristic: Look up package location (L) in your database $\Rightarrow$ 2 moves needed!

1.3 Key Insight: Why Bigger Patterns = Better (But Slower)

The Pattern Size Trade-off

Small pattern like $P = \{\text{package}\}$:

+ Few abstract states (4 states: L, R, A, B)
+ Tiny database, instant lookups
+ Quick to precompute
− Weak heuristic (ignores where trucks are!)

Medium pattern like $P = \{\text{package, truck_A}\}$:

+ More abstract states (8 states: 4 package $\times$ 2 truck_A)
+ Stronger heuristic (knows if truck_A can help!)
− Bigger database
− More precomputation

Huge pattern like $P = \{\text{package, truck_A, truck_B}\}$:

+ Perfect heuristic (this IS the full problem!)
− 16 states - same as original problem!
− **Defeats the purpose!**

Sweet spot: Pick enough variables to be informative, but few enough that the abstract problem is MUCH easier than the original.

1.4 The Hash Function: How to Actually Look Things Up

Making Lookups Fast

Problem: You have pattern $P = \{\text{package, truck_A}\}$ with values like (package=A, truck_A=L).
How do you quickly find this in a table?

Solution: Convert state to a single number (the hash)!

The Formula:
Assign numbers to values:

- package: L=0, R=1, A=2, B=3 (4 values)
- truck_A: L=0, R=1 (2 values)

Hash formula:

$$\text{index} = (\text{package value}) \times 1 + (\text{truck_A value}) \times 4$$

Example: (package=A, truck_A=L) $\Rightarrow$ index $= 2 \times 1 + 0 \times 4 = 2$

Now just look up position 2 in your table! Array access $= O(1)$.

Why it works: This is like counting in mixed-radix. First variable changes fastest, second variable changes every 4 steps. Gives unique index for every state combination.

1.5 Real Example: Package Delivery

Setup: Simple Delivery Problem

The Scenario:

- 2 locations: Left (L) and Right (R)
- 2 trucks: Truck A and Truck B
- 1 package starts at L
- **Goal:** Get package to R

What You Can Do:

- `pickup(truck, location)`: Truck picks up package if both are at same location
- `drop(truck, location)`: Truck drops package if both at same location
- `move(truck, from, to)`: Truck drives from one location to another

Initial Setup: Package at L, both trucks at R
Full State: (package=L, truck_A=R, truck_B=R)

This gives us $4 \times 2 \times 2 = 16$ possible states total. Too many? Let's abstract!

Pattern 1: Just Track the Package

Pattern Choice: $P = \{\text{package}\}$ (ignore both trucks)

What Happens:

- Package can be at: L, R, inside_A, inside_B (4 states instead of 16!)
- `move` actions? They don't affect the package, so they become self-loops
- `pickup`/`drop`? Keep these—they move the package

Solving the Abstract Problem:
Start from goal (package at R) and work backwards:

Package Location	Min Moves to Goal
R (goal)	0 moves
Inside A or B	1 move (just drop at R)
L	2 moves (pickup to truck, then drop at R)

Using It:
Initial state: (package=L, truck_A=R, truck_B=R)
Extract pattern: package=L
Lookup in database: **2 moves minimum!**

Why this is admissible: The real solution needs AT LEAST 2 moves for the package (even if trucks need more moves to get in position). We're giving a lower bound.

Pattern 2: Track Package + Truck A

Pattern Choice: $P = \{\text{package, truck_A}\}$ (ignore truck_B)

What Happens:

- Now we have $4 \times 2 = 8$ abstract states (package $\times$ truck_A position)
- Actions involving truck_B? Still become self-loops
- But now we track if truck_A can help!

The Hash Function (How to Index):
State: (package=inside_A, truck_A=L)
Convert to numbers: package=2, truck_A=0
Index $= 2 \times 1 + 0 \times 4 = 2$
Look up position 2 in table!

The Database:

State	(Pkg,TruckA)	Meaning	Index	h-value
LL	(L, L)	Both at L	0	2
RL	(R, L)	Pkg at R, truck at L	1	0 (goal!)
AL	(A, L)	Pkg in A, A at L	2	2
BL	(B, L)	Pkg in B, B at L	3	1
LR	(L, R)	Both at R	4	2
RR	(R, R)	Both at R	5	0 (goal!)
AR	(A, R)	Pkg in A, A at R	6	1
BR	(B, R)	Pkg in B, B at R	7	1

Better Heuristic!
State (Pkg=inside_A, truck_A=R) gets h=1 instead of h=1 from pattern 1.
But State (Pkg=L, truck_A=L) gets h=2 (same as pattern 1 for this case).
Pattern 2 dominates Pattern 1: Always $\geq$, sometimes strictly better!

2 Pathological Abstractions

2.1 The Trade-off Challenge

Conflicting Objectives

The eternal trade-off between accuracy and efficiency:

- **We want an informative heuristic** (high accuracy)
- **We want small computational overhead** (high efficiency)

The abstraction function α is a very powerful parameter, allowing us to travel the whole way between both extremes.

What Does "Small Computational Overhead" Mean?

Two key requirements:

1. **Fast computation of α:** For a given state s, the abstract state $\alpha(s)$ must be efficiently obtainable.
2. **Few abstract states:** For a given abstract state $\alpha(s)$, the heuristic value $h^\alpha(s) = h^*_{\Theta^\alpha}(\alpha(s))$ must be efficiently computable.

   Breaking down this formula:

   - $h^\alpha(s)$: The heuristic value for concrete state s using abstraction α
   - $\alpha(s)$: The abstract state that s maps to under abstraction α
   - Θ^α: The abstract state space induced by abstraction α
   - $h^*_{\Theta^\alpha}(\alpha(s))$: The optimal cost from abstract state $\alpha(s)$ to the nearest goal in the abstract space Θ^α

   In words: To compute the heuristic for state s, we first map s to its abstract state $\alpha(s)$, then look up the optimal cost from that abstract state to a goal in the abstract state space.

2.2 Pathological Case 1: One-State Abstraction

The Trivial Abstraction

Definition: $\alpha(s) :=$ const for all states s.

This means ALL concrete states map to a SINGLE abstract state.

Advantages:

+ Trivial to compute $\alpha(s)$ (constant time)
+ Trivial to compute $h^*_{\Theta^\alpha}$ (just one abstract state)
+ Minimal space requirements (store one value)

Disadvantages:

– **Completely uninformative** h^α
– $h^\alpha(s) = 0$ for all states (if initial = goal) or constant otherwise
– No guidance for search

Visualization

In the "Logistics mal anders" example, the one-state abstraction merges all 16 concrete states into a single abstract state. The abstract state space consists of just one node, making the heuristic useless for guiding search.

2.3 Pathological Case 2: Identity Abstraction

The Perfect Accuracy Abstraction

Definition: $\alpha(s) := s$ for all states s.

This means each concrete state maps to itself as an abstract state.

Advantages:

+ $h^\alpha = h^*$ (perfect heuristic!)
+ Trivial to compute $\alpha(s)$ (identity function)
+ Maximally informative

Disadvantages:

– **Computing $h^*_{\Theta^\alpha}$ = solving the optimal planning problem**
– Abstract state space = concrete state space
– No computational advantage whatsoever
– Defeats the entire purpose of abstraction

The Fundamental Issue

While the identity abstraction gives us perfect heuristic values, computing them requires solving the original problem. This is equivalent to using blind search with no heuristic guidance.

Lesson: We need abstractions that strike a balance—neither too coarse (one-state) nor too fine (identity).

2.4 Pathological Case 3: Perfect Abstraction

The h* Abstraction

Definition: $\alpha(s) := h^*(s)$ for all states s.

This means states are grouped by their optimal cost to the goal.

Advantages:

+ $h^\alpha = h^*$ (perfect heuristic!)
+ Few abstract states (typically $\ll |S|$)
+ Easy to compute $h^*_{\Theta^\alpha}$ once we have the abstract states
+ Each abstract state is just a distance value

Disadvantages:

– **Computing $\alpha(s) = h^*(s)$ = solving the optimal planning problem**
– We need to know $h^*(s)$ to compute the abstraction
– Circular dependency: need heuristic to compute heuristic

The Irony

The perfect abstraction groups states by their optimal cost. While this gives few abstract states and perfect heuristic values, computing which group a state belongs to requires solving the very problem we're trying to solve!

Example: In "Logistics mal anders", states would be grouped as:

- Group 0: All goal states (RRR, RRL, RLR, RLL)
- Group 1: States 1 step from goal
- Group 2: States 2 steps from goal
- Group 3: States 3 steps from goal
- Group 4: The initial state LRR

But determining which group a state belongs to requires computing $h^*(s)$!

2.5 Summary of Pathological Cases

The Abstraction Spectrum

Abstraction Type	Compute α	Compute $h^*_{\Theta^\alpha}$	Informativeness
One-State	Trivial	Trivial	Useless
Identity	Trivial	NP-hard	Perfect
Perfect	NP-hard	Easy	Perfect
Practical PDB	**Fast**	**Polynomial**	**Good**

The Goal: Find abstractions that are:

- Fast to compute (polynomial in $\|\Pi\|$)
- Have manageable abstract state spaces (polynomial in $\|\Pi\|$)
- Provide informative heuristic values (close to h^*)

How to Obtain Non-Pathological Abstractions

Covered in this course:

- **Pattern Database (PDB) Heuristics:** Projections to subsets of variables

Not covered (but exist):

- **Domain Abstractions:** Aggregate values within variable domains
- **Cartesian Abstractions:** Cross-products of domain-subsets
- **Merge-and-Shrink Abstractions:** Iterative merging and shrinking

3 Orthogonal Pattern Databases

3.1 Motivation: Combining Multiple PDBs

The Limitation of Single PDBs

Problem: A single PDB with a small pattern P might be too weak (underestimates too much)

Example: In logistics, tracking only the package location ignores truck positions, leading to significant underestimation.

Idea: Use *multiple* PDBs and **combine** them!

Question: Can we simply add heuristic values from different patterns?
Answer: NOT ALWAYS! Summing arbitrary PDBs can **overestimate** (lose admissibility).

Solution: Use **orthogonal patterns** (patterns that don't interfere with each other).

3.2 Maximizing vs. Summing Heuristics

Two Approaches to Combining Heuristics

Approach 1: Maximization (Always Safe)

- Given heuristics h^{P_1} and h^{P_2}
- Compute $h(s) = \max\{h^{P_1}(s), h^{P_2}(s)\}$
- Always admissible if both are admissible
- Dominates each individual heuristic
- Often provides modest improvement

Approach 2: Summation (Conditionally Safe)

- Given heuristics h^{P_1} and h^{P_2}
- Compute $h(s) = h^{P_1}(s) + h^{P_2}(s)$
- Only admissible under certain conditions
- Can provide much higher estimates than maximization
- **Requires orthogonality!**

3.3 Actions Affecting Abstractions

Self-Loop

A transition $(s, a, t) \in T$ is a **self-loop** if $s = t$.

In an abstract state space, a transition $(\alpha(s), a, \alpha(t)) \in T^\alpha$ is a self-loop if $\alpha(s) = \alpha(t)$.

Actions Affecting an Abstraction

Let α be an abstraction of Θ, and let a be an action in Θ.

We say that a **affects** α if Θ^α has at least one **non-self-loop** transition induced by a.

Equivalently: $\exists$ transition $(\alpha(s), a, \alpha(t))$ with $\alpha(s) \neq \alpha(t)$.

Intuition: What Does "Affecting" Mean?

An action affects an abstraction if it can make a "real change" in the abstract state space.

Example: In the package-only abstraction:

- `pickup(A, L)` affects the abstraction (changes package location)
- `move(A, L, R)` does NOT affect the abstraction (truck movement is invisible)

Actions that don't affect an abstraction only create self-loops in the abstract space.

3.4 Orthogonal Abstractions: The Core Definition

Orthogonal Abstractions

Let α_1 and α_2 be abstractions of Θ.

We say that α_1 and α_2 are **orthogonal** (written $\alpha_1 \perp \alpha_2$) if:

No action of Θ affects both α_1 and α_2

Formally: For all actions $a \in A$, if a affects α_1, then a does not affect α_2, and vice versa.

Why Orthogonality Matters

The Key Insight: "Self-loops don't count"

When abstractions are orthogonal:

- Every action affects **at most one** of the abstractions
- Actions counted in h^{α_1} are NOT counted in h^{α_2}
- No "double counting" of action costs
- Therefore: $h^{\alpha_1} + h^{\alpha_2}$ remains admissible!

Intuition: In an optimal solution path, each action contributes to at most one abstraction's cost estimate, so summing the estimates gives a lower bound on the total cost.

3.5 Orthogonal Patterns in FDR Tasks

Variable Affectedness

Let $a \in A$ be an action in an FDR task.

A variable $v \in V$ is **affected by** a if:

$$(\text{eff}_a)(v) \neq \bot$$

In words: v is affected if a changes the value of v.

We denote:

$$\text{aff}(a) = \{v \in V : v \text{ is affected by } a\}$$

This is the set of all variables that action a modifies.

Actions Affecting PDBs - FDR Version

Let P be a pattern for an FDR planning task Π, and let a be an action in Π.

Then a affects π_P **if and only if** there exists a variable $v \in P$ on which eff_a is defined.

Equivalently: a affects $\pi_P \Leftrightarrow \text{aff}(a) \cap P \neq \emptyset$

Orthogonal Patterns

Let $P_1, P_2 \subseteq V$ be two patterns.

They are **orthogonal** (written $P_1 \perp P_2$) if:

$$\text{For all actions } a \in A: \quad |\text{aff}(a) \cap (P_1 \cup P_2)| \leq 1$$

In words: Every action affects **at most one** variable from $P_1 \cup P_2$.

Equivalent condition: For all actions $a \in A$:

$$\text{aff}(a) \cap P_1 \neq \emptyset \quad \Rightarrow \quad \text{aff}(a) \cap P_2 = \emptyset$$

No action simultaneously affects variables from both patterns.

3.6 Understanding Orthogonality: Key Questions

Common Misconceptions About Orthogonality

Question 1: If $P_1 \cap P_2 = \emptyset$ (patterns don't share variables), are P_1 and P_2 necessarily orthogonal?

Answer: NO!

Counterexample: Consider patterns $P_1 = \{A\}$ and $P_2 = \{B\}$ where:

- $P_1 \cap P_2 = \emptyset$ (disjoint patterns)
- There exists action `fancyCar` with effects on both A and B
- Then `fancyCar` affects both P_1 and P_2
- Therefore P_1 and P_2 are **not orthogonal**

Lesson: Disjoint patterns are not sufficient for orthogonality. We must check that no action has "cross-effects."

Another Misconception

Question 2: If $P_1 \cap P_2 \neq \emptyset$ (patterns share variables), can P_1 and P_2 be orthogonal?

Answer: Technically yes, but pathological

The only way this works is if all variables in $P_1 \cap P_2$ are **never affected by any action**, which is highly pathological and useless in practice.

Practical Rule: For non-pathological cases, orthogonal patterns must be disjoint.

But remember: Disjoint $\not\Rightarrow$ Orthogonal (as shown above)!

3.7 Examples of Orthogonal and Non-Orthogonal Patterns

Example 1: 15-Puzzle (Orthogonal)

15-Puzzle State Variables: $V = \{x_1, x_2, \ldots, x_{15}\}$ (positions of tiles)

Pattern 1: $P_1 = \{x_1, x_2, x_3, x_4\}$ (tiles 1-4)
Pattern 2: $P_2 = \{x_5, x_6, x_7, x_8\}$ (tiles 5-8)

Are P_1 and P_2 orthogonal?
Check: Each move action swaps the blank with one tile

- Moving tile 1: affects only $x_1 \in P_1$
- Moving tile 5: affects only $x_5 \in P_2$
- Moving the blank: affects at most one variable from $P_1 \cup P_2$
- No action affects both a variable from P_1 and a variable from P_2

YES! $P_1 \perp P_2$ because each action affects at most one tile.

Important Detail: If we include the blank position as a variable in either pattern, they would NO LONGER be orthogonal! (The blank position changes with every move affecting tiles from either pattern)

Example 2: Logistics "mal anders"

Variables: $V = \{p, t_A, t_B\}$ where:

- p = package location
- t_A = truck A location
- t_B = truck B location

Pattern 1: $P_1 = \{p\}$ (package only)
Pattern 2: $P_2 = \{t_A\}$ (truck A only)

Actions and their affectedness:

- pickup(A, L): affects p only $\Rightarrow$ aff $\cap(P_1 \cup P_2) = \{p\}$ (size 1) ✓
- drop(A, L): affects p only $\Rightarrow$ aff $\cap(P_1 \cup P_2) = \{p\}$ (size 1) ✓
- move(A, L, R): affects t_A only $\Rightarrow$ aff $\cap(P_1 \cup P_2) = \{t_A\}$ (size 1) ✓
- Actions involving truck B: affect only p or t_B (not in P_2)

Orthogonal! Each action affects at most one variable from $P_1 \cup P_2$.

Example 3: Attack of the Zombie Tomatoes (Non-Orthogonal)

Goal: A and B both true.
Initial state: A and B both false.
Actions:

- `carA`: effect A, cost 2
- `carB`: effect B, cost 2
- `fancyCar`: effect A **and** B, cost 3

Patterns: $P_1 = \{A\}$ and $P_2 = \{B\}$

Are P_1 and P_2 orthogonal?
NO! Action `fancyCar` affects both patterns:

- $\text{aff}(\texttt{fancyCar}) = \{A, B\}$
- $\text{aff}(\texttt{fancyCar}) \cap P_1 = \{A\} \neq \emptyset$
- $\text{aff}(\texttt{fancyCar}) \cap P_2 = \{B\} \neq \emptyset$
- Therefore $|\text{aff}(\texttt{fancyCar}) \cap (P_1 \cup P_2)| = 2 > 1$

Consequence:

- $h^{P_1}(I) = 2$ (using `carA`)
- $h^{P_2}(I) = 2$ (using `carB`)
- $h^*(I) = 3$ (using `fancyCar` is optimal)
- If we sum: $h^{P_1}(I) + h^{P_2}(I) = 4 > 3 = h^*(I)$ (INADMISSIBLE!)
- We must use max instead: $\max\{h^{P_1}(I), h^{P_2}(I)\} = 2 < 3 = h^*(I)$ ✓

Why the overestimate? The action `fancyCar` achieves both goals simultaneously, but both patterns count it (or equivalent actions) separately, leading to double-counting.

3.8 The Additivity Theorem

Theorem: Orthogonal Abstractions are Additive

Let $\alpha_1, \alpha_2, \ldots, \alpha_k$ be **pairwise orthogonal abstractions** for the same state space Θ:

$$\alpha_i \perp \alpha_j \quad \text{for all } i \neq j$$

Define the **additive heuristic**:

$$h^{\{\alpha_1,\ldots,\alpha_k\}}(s) = \sum_{i=1}^{k} h^{\alpha_i}(s)$$

Then: $h^{\{\alpha_1,\ldots,\alpha_k\}}$ is **consistent**, and therefore also **admissible and safe**!

Proof Sketch of Additivity Theorem

Key Idea: Orthogonality implies that actions can be "partitioned" according to which abstraction they affect.

Proof:
Let $\pi = \langle a_1, a_2, \ldots, a_n \rangle$ be an optimal plan from s to goal with cost $h^*(s)$.

Step 1: Partition the plan
For each abstraction α_i, let:

$$\pi_i = \langle a_j \in \pi : a_j \text{ affects } \alpha_i \rangle$$

These are the actions in π that affect abstraction α_i.

Step 2: Use orthogonality
Since abstractions are pairwise orthogonal:

$$a_j \text{ affects } \alpha_i \quad \text{and} \quad a_j \text{ affects } \alpha_\ell \quad \Rightarrow \quad i = \ell$$

Therefore: $\{\pi_1, \pi_2, \ldots, \pi_k\}$ is a **partition** of π (disjoint sets that cover all actions).

Step 3: Cost decomposition

$$\text{cost}(\pi) = \sum_{i=1}^{k} \text{cost}(\pi_i)$$

Step 4: Lower bound for each abstraction
For each i, the sequence π_i induces a path in Θ^{α_i} from $\alpha_i(s)$ to a goal state.
Therefore:

$$h^{\alpha_i}(s) \leq \text{cost}(\pi_i)$$

Step 5: Sum over all abstractions

$$\sum_{i=1}^{k} h^{\alpha_i}(s) \leq \sum_{i=1}^{k} \text{cost}(\pi_i) = \text{cost}(\pi) = h^*(s)$$

Therefore: $h^{\{\alpha_1,\ldots,\alpha_k\}}(s) \leq h^*(s)$ ✓(admissible!)

Consistency follows by a similar argument applied to individual transitions. □

3.9 Orthogonal Patterns are Additive

Theorem: Orthogonal Patterns are Additive

Let $\{P_1, \ldots, P_k\}$ be an **orthogonal pattern collection** for an FDR planning task (i.e., $P_i \perp P_j$ for all $i \neq j$).

Then:

$$h^{\{P_1,\ldots,P_k\}}(s) = \sum_{i=1}^{k} h^{P_i}(s)$$

is **consistent**, and therefore also **admissible and safe**.

Proof: The abstractions $\pi_{P_1}, \ldots, \pi_{P_k}$ are pairwise orthogonal. The rest follows from the general theorem above. □

Terminology

Instead of saying "collection of pairwise orthogonal patterns," we also say:

- **Orthogonal pattern collection**, or
- **Orthogonal collection**

Important Domination Result

Let $\{P_1, \ldots, P_k\}$ be an orthogonal pattern collection for an FDR planning task, and let P be a pattern such that $P_i \subseteq P$ for all $i \in \{1, \ldots, k\}$.

Then:

$$\sum_{i=1}^{k} h^{P_i} \leq h^P$$

Intuition: A larger pattern P that contains all smaller patterns gives at least as good a heuristic value as summing the smaller patterns' values.

Proof Sketch: Each π_{P_i} is a projection of $\Pi|_P$. The collection $\{P_1, \ldots, P_k\}$ is orthogonal in $\Pi|_P$ as well. Admissibility of $\sum_{i=1}^{k} h^{P_i}$ in $\Pi|_P$ gives us the result.

3.10 Pattern Collections and The Cliques Heuristic

Pattern Collections

A **pattern collection** $\mathcal{C}$ is simply a set of patterns:

$$\mathcal{C} = \{P_1, P_2, \ldots, P_m\}$$

where each $P_i \subseteq V$ is a pattern.

Why use pattern collections?

- A single PDB is limited by memory (exponential in pattern size)
- Multiple small PDBs can be more informative when combined
- Different patterns capture different aspects of the problem

The Compatibility/Orthogonality Graph

Given a pattern collection $\mathcal{C} = \{P_1, P_2, \ldots, P_m\}$, the **orthogonality graph** (or **compatibility graph**) $G = (V, E)$ is defined as:

- **Vertices:** $V = \mathcal{C}$ (each pattern is a vertex)
- **Edges:** $(P_i, P_j) \in E$ if and only if $P_i \perp P_j$ (edge if orthogonal)

A **clique** in this graph is a set of mutually orthogonal patterns.

A **maximal clique** is a clique that cannot be extended by adding any other pattern.

The Cliques Heuristic

Let $\mathcal{C} = \{P_1, P_2, \ldots, P_m\}$ be a pattern collection.
Let cliques($\mathcal{C}$) be the set of all maximal cliques in the orthogonality graph for $\mathcal{C}$.

The **cliques heuristic** is:

$$h_{\mathcal{C}}^{\text{cliques}}(s) = \max_{D \in \text{cliques}(\mathcal{C})} \sum_{P \in D} h^P(s)$$

In words: For each maximal clique of orthogonal patterns, compute the sum of PDB values. Take the maximum over all such sums.

Properties:

- Admissible and consistent (max of admissible heuristics)
- Dominates all individual PDBs: $h_{\mathcal{C}}^{\text{cliques}}(s) \geq h^P(s)$ for all $P \in \mathcal{C}$
- Often much stronger than individual PDBs
- Best possible heuristic from $\mathcal{C}$ using only orthogonality

3.11 Computing the Cliques Heuristic

Algorithm: How to Exploit Orthogonal Patterns

Given pattern collection $\mathcal{C}$: How to get the best possible lower bound?

Step 1: Build the orthogonality graph for $\mathcal{C}$

- Vertices are the patterns $P \in \mathcal{C}$
- Add edge between pairs of orthogonal patterns

Step 2: Compute all maximal cliques

- Find all maximal orthogonal sub-collections
- Note: Computing maximal cliques is NP-hard in general
- But the orthogonality graph is typically small, so this is feasible

Step 3: Compute the cliques heuristic
For each state s during search:

$$h_{\mathcal{C}}^{\text{cliques}}(s) = \max_{D \in \text{cliques}(\mathcal{C})} \sum_{P \in D} h^P(s)$$

This involves:

1. For each maximal clique D:
   - Sum the PDB values for all patterns in D
2. Take the maximum over all such sums

Complexity Considerations

Precomputation:

- Computing each PDB: polynomial in $\|\Pi\| + \|\Theta^P\|$
- Building orthogonality graph: $O(m^2 \cdot |A|)$ where $m = |\mathcal{C}|$
- Finding maximal cliques: NP-hard, but feasible for small graphs

During search (per state):

- Looking up each PDB value: $O(|P|)$ per pattern
- Summing over clique: $O(|\mathcal{C}|)$ per clique
- Taking max over cliques: $O(|\text{cliques}(\mathcal{C})|)$
- Total: $O(|\mathcal{C}| \cdot |\text{cliques}(\mathcal{C})|)$ per state

3.12 Worked Example: Simple Pattern Collection

Abstract Example with 3 Variables

FDR task: $V = \{v_1, v_2, v_3\}$
Pattern collection: $\mathcal{C} = \{P_1, P_2, P_3, P_4\}$ where:

- $P_1 = \{v_1, v_2\}$
- $P_2 = \{v_1\}$
- $P_3 = \{v_2\}$
- $P_4 = \{v_3\}$

Actions:

- There are actions affecting each individual variable
- The **only** actions affecting several variables affect v_1 and v_3
- No action affects both v_2 and v_3 simultaneously

Step 1: Determine orthogonality
Check each pair:

- $P_1 \perp P_2$? NO ($v_1 \in P_1 \cap P_2$, both affected by actions on v_1)
- $P_1 \perp P_3$? NO ($v_2 \in P_1 \cap P_3$, both affected by actions on v_2)
- $P_1 \perp P_4$? NO (actions affect both $v_1 \in P_1$ and $v_3 \in P_4$)
- $P_2 \perp P_3$? YES (no action affects both v_1 and v_2 only)
- $P_2 \perp P_4$? NO (actions affect both v_1 and v_3)
- $P_3 \perp P_4$? YES (no action affects both v_2 and v_3)

Step 2: Build orthogonality graph
Edges: (P_2, P_3) and (P_3, P_4)

Step 3: Find maximal cliques
Maximal cliques in the orthogonality graph:

- $\{P_1\}$ (singleton, cannot be extended)
- $\{P_2, P_3\}$ (cannot add P_1 or P_4)
- $\{P_3, P_4\}$ (cannot add P_1 or P_2)

Step 4: Cliques heuristic

$$h^{\mathcal{C}}(s) = \max\{h^{P_1}(s),\ h^{P_2}(s) + h^{P_3}(s),\ h^{P_3}(s) + h^{P_4}(s)\}$$

This is the best admissible heuristic we can derive from $\mathcal{C}$ using only orthogonality information!

3.13 Example: Australia Driving Problem

Australia Example

Variables:

- at : $\{\text{Sy}, \text{Ad}, \text{Br}, \text{Pe}, \text{Da}\}$ (current location)
- $v(x) : \{T, F\}$ for $x \in \{\text{Sy}, \text{Ad}, \text{Br}, \text{Pe}, \text{Da}\}$ (visited cities)

Actions: drive(x, y) where x, y have a road
Costs: Sy $\leftrightarrow$ Br: 1, Sy $\leftrightarrow$ Ad: 2, Ad $\leftrightarrow$ Pe: 3, Ad $\leftrightarrow$ Da: 4
Initial state: at $=$ Sy, $v(\text{Sy}) = T$, $v(x) = F$ for $x \neq \text{Sy}$
Goal: at $=$ Sy, $v(x) = T$ for all x

Observation 1: Patterns $P_1 = \{v(\text{Br})\}$ and $P_2 = \{v(\text{Pe}), v(\text{Da})\}$
Are P_1 and P_2 orthogonal?
YES! Every action affects at most one of them.

- Driving to Brisbane only affects $v(\text{Br})$
- Driving to Perth/Darwin only affects $v(\text{Pe})$ or $v(\text{Da})$
- No action affects variables from both patterns

PDB values:

- $h^{P_1}(I) = 1$ (must visit Brisbane: Sy $\rightarrow$ Br)
- $h^{P_2}(I) = 4 + 3 = 7$ (must visit Perth and Darwin via Adelaide)

Cliques heuristic: $h^{\mathcal{C}}(I) = h^{P_1}(I) + h^{P_2}(I) = 1 + 7 = 8$

Observation 2: Pattern $P_1' = \{v(\text{Br}), \text{at}\}$ and $P_2 = \{v(\text{Pe}), v(\text{Da})\}$
Are P_1' and P_2 orthogonal?
NO! Driving to Perth affects both patterns:

- Changes `at` variable (in P_1')
- Changes $v(\text{Pe})$ (in P_2)

PDB values:

- $h^{P_1'}(I) = 2$ (visit Brisbane and return: Sy $\rightarrow$ Br $\rightarrow$ Sy)
- $h^{P_2}(I) = 7$ (same as before)

Cliques heuristic: $h^{\mathcal{C}}(I) = \max\{h^{P_1'}(I), h^{P_2}(I)\} = \max\{2, 7\} = 7$

Note: Even though P_1' gives a better value than P_1 individually (2 vs 1), replacing P_1 with P_1' makes the overall heuristic worse (7 vs 8) because we lose the ability to add!

3.14 Limitations of the Cliques Heuristic

The Cliques Heuristic is Not Perfect

The cliques heuristic $h^{\mathcal{C}}$ is the **best possible admissible heuristic** we can derive from $\mathcal{C}$ **based on exploiting only orthogonality**.

However, it may still underestimate significantly!

Attack of the Zombie Tomatoes Revisited

Recall the problem:

- Goal: A and B both true
- Initial state: A and B both false
- Actions:
  - `carA`: effect A, cost 2
  - `carB`: effect B, cost 2
  - `fancyCar`: effect A and B, cost 3
- Patterns: $P_1 = \{A\}$ and $P_2 = \{B\}$

Analysis:

- P_1 and P_2 are **not orthogonal** (due to `fancyCar`)
- $h^{P_1}(I) = 2$, $h^{P_2}(I) = 2$
- Maximal cliques: $\{P_1\}$ and $\{P_2\}$ (singletons)
- $h^{\mathcal{C}}(I) = \max\{2, 2\} = 2$
- But $h^*(I) = 3$ (using `fancyCar`)

The gap: $h^{\mathcal{C}}(I) = 2 < 3 = h^*(I)$

Why the underestimate? The cliques heuristic cannot capture the synergy of `fancyCar` achieving both goals together.

Beyond Orthogonality: Cost Partitioning

Question: Can we combine PDBs that are *not* orthogonal and still maintain admissibility?
Answer: YES! Using **cost partitioning**.

Idea: Instead of requiring patterns to be orthogonal, we "partition" the action costs among patterns.

Optimal Cost Partitioning (OCP): For a pattern collection $\{P_1, \ldots, P_k\}$, find cost functions $\text{cost}_1, \ldots, \text{cost}_k$ such that:

$$\sum_{i=1}^{k} \text{cost}_i(a) \leq \text{cost}(a) \quad \text{for all } a \in A$$

Then compute:

$$h^{\text{OCP}}(s) = \max_{\text{valid partitions}} \sum_{i=1}^{k} h^{P_i}_{\text{cost}_i}(s)$$

Properties:

- Always admissible and consistent
- Dominates cliques heuristic
- Can achieve $h^{\text{OCP}}(I) = 3$ in Zombie Tomatoes example
- Computing OCP requires solving an LP for each state (expensive!)

Not covered in detail in this course, but important for advanced applications.

4 Summary and Key Takeaways

4.1 Main Concepts Summary

Pathological Abstractions

Three extremes to avoid:

1. **One-State Abstraction:** All states map to one abstract state
   - Fast but completely uninformative
2. **Identity Abstraction:** Each state maps to itself
   - Perfect heuristic but NP-hard to compute
3. **Perfect Abstraction:** States grouped by h^* value
   - Perfect heuristic but computing α requires solving the problem

Goal: Find practical abstractions that balance informativeness and efficiency

Orthogonal Patterns

Key Definitions:

- Action a **affects** pattern P if $\text{aff}(a) \cap P \neq \emptyset$
- Patterns P_1, P_2 are **orthogonal** if no action affects both
- Formally: $|\text{aff}(a) \cap (P_1 \cup P_2)| \leq 1$ for all $a \in A$

Main Theorem: Orthogonal patterns are additive

$$h^{\{P_1,\ldots,P_k\}}(s) = \sum_{i=1}^{k} h^{P_i}(s) \text{ is admissible}$$

Important Facts:

- Disjoint patterns $\not\Rightarrow$ orthogonal (beware cross-effects!)
- Overlapping patterns are (almost) never orthogonal
- Orthogonality enables summing, which is much stronger than max

The Cliques Heuristic

Construction:

1. Build orthogonality graph for pattern collection $\mathcal{C}$
   - Vertices = patterns
   - Edges = orthogonality relations
2. Find all maximal cliques
   - Each clique = maximal set of mutually orthogonal patterns
3. For each state, compute:

$$h^{\mathcal{C}}(s) = \max_{D \in \text{cliques}} \sum_{P \in D} h^P(s)$$

Properties:

- Admissible and consistent
- Dominates all individual PDBs in $\mathcal{C}$
- Best possible heuristic from $\mathcal{C}$ using only orthogonality
- May still underestimate (see Zombie Tomatoes example)

Going Beyond: Cost partitioning can combine non-orthogonal patterns while maintaining admissibility, but is more expensive to compute.

4.2 Quick Reference Guide

Checking Orthogonality: Step-by-Step

Given: Patterns $P_1, P_2 \subseteq V$

Step 1: Quick check - Are patterns disjoint?

- If $P_1 \cap P_2 \neq \emptyset$: Almost certainly NOT orthogonal (except pathological cases)
- If $P_1 \cap P_2 = \emptyset$: Proceed to Step 2

Step 2: For each action $a \in A$:

1. Compute $\text{aff}(a) = \{v \in V : \text{eff}_a(v) \neq \bot\}$
2. Check if $|\text{aff}(a) \cap (P_1 \cup P_2)| \leq 1$
3. If any action violates this: NOT orthogonal

Step 3: If all actions pass Step 2: Orthogonal! ✓

Common Pitfall: Don't assume disjoint $\Rightarrow$ orthogonal! Always check for cross-effects.

Common Mistakes to Avoid

Mistake 1: Adding Non-Orthogonal PDBs

- **Wrong:** $h(s) = h^{P_1}(s) + h^{P_2}(s)$ when P_1 and P_2 are not orthogonal
- **Fix:** Check orthogonality first! Use max if not orthogonal

Mistake 2: Assuming Disjoint = Orthogonal

- **Wrong:** "Patterns don't share variables, so they must be orthogonal"
- **Fix:** Check for actions with cross-effects!

Mistake 3: Including Blank/Position Variables Carelessly

- **Wrong:** Including blank position in both 15-puzzle patterns
- **Fix:** Exclude variables that are affected by all actions affecting either pattern

Mistake 4: Forgetting to Compute Maximal Cliques

- **Wrong:** Just using pairwise orthogonal patterns
- **Fix:** Find ALL maximal cliques for best results

4.3 Practice Problems

Exercises

Problem 1: Identifying Pathologies
For each abstraction below, classify it as one-state, identity, perfect, or practical:

(a) $\alpha(s) =$ number of variables different from goal

(b) $\alpha(s) = 42$ for all states s

(c) $\alpha(s) = (s(v_1), s(v_2))$ where $V = \{v_1, v_2, \ldots, v_{10}\}$

(d) $\alpha(s) = h^*(s)$

Problem 2: Checking Orthogonality
Consider a Blocksworld domain with 4 blocks $\{A, B, C, D\}$.
State variables: $\text{on}(x)$ for each block (what is block x on?)
Patterns: $P_1 = \{\text{on}(A), \text{on}(B)\}$ and $P_2 = \{\text{on}(C), \text{on}(D)\}$
Are P_1 and P_2 orthogonal? Justify your answer.

Problem 3: Computing Cliques Heuristic
Given pattern collection $\mathcal{C} = \{P_1, P_2, P_3, P_4\}$ with orthogonality:

- $P_1 \perp P_3$
- $P_2 \perp P_4$
- All other pairs are not orthogonal

And PDB values: $h^{P_1}(s) = 5$, $h^{P_2}(s) = 3$, $h^{P_3}(s) = 7$, $h^{P_4}(s) = 4$
Compute $h^{\mathcal{C}}(s)$.

Problem 4: Zombie Tomatoes with Cost Partitioning
In the Zombie Tomatoes example, show that a cost partitioning with:

- $\text{cost}_1(\texttt{fancyCar}) = 2$, $\text{cost}_1(\texttt{carA}) = 2$, $\text{cost}_1(\texttt{carB}) = 0$
- $\text{cost}_2(\texttt{fancyCar}) = 1$, $\text{cost}_2(\texttt{carA}) = 0$, $\text{cost}_2(\texttt{carB}) = 2$

gives $h(I) = 3$ (the optimal value).

Problem 5: 15-Puzzle Analysis
For the 15-puzzle, consider three patterns:

- $P_1 = \{x_1, x_2, x_3, x_4\}$
- $P_2 = \{x_5, x_6, x_7, x_8\}$
- $P_3 = \{x_9, x_{10}, x_{11}, x_{12}\}$

(a) Are these patterns pairwise orthogonal (excluding the blank)?

(b) What is the size of each PDB?

(c) If we include the blank position as a variable in P_1, are P_1 and P_2 still orthogonal?

4.4 Further Reading and Advanced Topics

- **Culberson & Schaeffer (1998):** *Pattern Databases* (original paper introducing PDBs)
- **Edelkamp (2001):** *Planning with Pattern Databases* (first application to planning)
- **Haslum et al. (2007):** *Domain-Independent Construction of Pattern Database Heuristics for Cost-Optimal Planning* (introduces cliques heuristic)
- **Pommerening et al. (2013):** *Optimal Planning with Pattern Database Heuristics* (cost partitioning methods)
- **Katz & Domshlak (2010):** *Optimal Admissible Composition of Abstraction Heuristics* (theoretical foundations)
- **Helmert et al. (2014):** *Merge & Shrink Abstraction* (more general abstraction framework)

End of PDB Pathologies & Orthogonal PDBs Guide

Master the art of combining pattern databases for powerful heuristics!